Theory Notes / version 0.2

Machine Learning A

Casper Moes (hwd362) 4. November 2026

Indhold

Generalization Bounds **5**

1 Concentration of measure inequalities **6**

1.1 Markow's inequality 6

1.2 Chebychew's inequality 6

1.3 Hoeffding's inequality 6

2 Generalization bounds: Finite and Countably Infinity Hypothesis Classes **8**

2.1 Single hypothesis 8

2.1.1 Setting and risk definitions 8

2.1.2 Theoretical properties of empirical risk 9

2.1.3 Sidenote: Understanding $\epsilon = \sqrt{\frac{1}{2N} \ln\left(\frac{1}{\delta}\right)}$ 11

2.2 Finite Hypothesis class 11

2.2.1 Hoeffdings bound does not apply anymore! 12

2.3 Lower Bound 15

2.4 Approximation and Estimation error 15

Supervised Learning **16**

3 Linear regression **17**

3.1 Linear Algebra prerequisites 17

3.1.1 Singular Value Decomposition 17

3.1.2 The Spectral Theorem 18

3.2 Overidentified case: $N > rank(X)$ 20

3.2.1 OLS is equivalent with Gaussian MLE 22

3.3 Underidentified case: $N < rank(X)$ 22

3.3.1 Minimum-norm solution 23

3.3.2 Singular Value Decomposition and the Pseudoinverse . . 24

3.3.3 Ridge regression (Two-norm regularization) 25

3.3.4 Equivalence theorem 25

3.3.5 Lasso regression (One-norm regularization) 27

3.4 Comment on double descent 27

4 Linear Classification **28**

4.1 Logistic regression 28

4.1.1 Binary case 28

4.1.2 Multiple classification 31

4.1.3 Optimization: Gradient descent methods 32

5 Random Forest **33**

5.1 Binary decision trees (definitions) 33

5.1.1 Regression trees 34

5.1.2 Classification trees . . . 34
5.1.3 Growing and pruning . . . 35
5.2 Ensembles and bagging . . . 35
5.3 Growing and evaluating a random forest . . . 36
5.4 Out-of-bag (OOB) error . . . 36
5.5 Discussion: CART vs Random Forest . . . 37

6 Neural Networks **37**
6.1 The Neuron . . . 37
6.1.1 Pre-Activation . . . 38
6.1.2 Activation and output . . . 38
6.1.3 Activation functions . . . 39
6.2 Gradient descent . . . 42
6.3 Backpropagation . . . 43
6.3.1 The ouput layer . . . 44
6.3.2 Hidden layers . . . 45

7 Deep Learning **45**
7.1 Convolution / cross-correlation . . . 45
7.1.1 Padding . . . 47
7.1.2 Equivariance . . . 47
7.2 Pooling layer . . . 47
7.2.1 Invariance . . . 48
7.2.2 Receptive field . . . 48

7.3 Example: VGG-16 architecture 49
7.4 Tricks-of-the-Trade 50
7.4.1 Ensemble Methods 50
7.4.2 Dropout 50
7.4.3 Residual Leraning / Shorcuts 51
7.4.4 Augmentation og Invariance 51
7.4.5 Batch and Layer Normalization 52

Unsupervised Learning **53**

8 Principal Component Analysis **54**
8.1 Definitions 54
8.1.1 Dimensions 54
8.2 The objective of PCA 54
8.3 Optimal reconstruction 55

I | Generalization Bounds

1 Concentration of measure inequalities

1.1 Markow's inequality

Statement: For any non-negative stochastic variable X and with $\epsilon > 0$ we have

$$\mathbb{P}(X \geq \epsilon) \leq \frac{\mathbb{E}[X]}{\epsilon}.$$

Proof: Define $Y := 1(X \geq \epsilon)$ then $Y \leq \frac{X}{\epsilon}$ (if $0 \leq X < \epsilon$ then $Y = 0 \leq \frac{X}{\epsilon}$ and if $X \geq \epsilon \iff \frac{X}{\epsilon} \geq 1$ then $Y = 1 \leq \frac{X}{\epsilon}$) therefore $\mathbb{P}(X \geq \epsilon) = \mathbb{P}(Y = 1) = \mathbb{E}[Y] \leq \mathbb{E}[\frac{X}{\epsilon}] = \frac{\mathbb{E}[X]}{\epsilon}$.

This can be rewritten by defining $a = \frac{\epsilon}{\mathbb{E}[X]}$ ($\mathbb{E}[X] \neq 0$) and we have

$$\mathbb{P}(X \geq a\mathbb{E}[X]) \leq \frac{1}{a}$$

. That is, the probability of an event a times away from the stochastic variables mean value is decaying with $1/a$.

1.2 Chebychew's inequality

Statement:

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq \epsilon) \leq \frac{\mathbb{V}(X)}{\epsilon^2}$$

Proof: For any $\epsilon > 0$ we have (by Markow's inequality) $\mathbb{P}((X - \mathbb{E}[X])^2 \geq \epsilon^2) \leq \frac{\mathbb{E}[(X-\mathbb{E}[X])^2]}{\epsilon^2} = \frac{\mathbb{V}(X)}{\epsilon^2}$. Finally we note that $|X - \mathbb{E}[X]| \geq \epsilon \iff (X - \mathbb{E}[X])^2 \geq \epsilon^2$ and thus $\mathbb{P}(|X - \mathbb{E}[X]| \geq \epsilon) \leq \frac{\mathbb{V}(X)}{\epsilon^2}$.

1.3 Hoeffding's inequality

Statement: Given the independent stochastic variables $X_1, \ldots, X_n$ where there exists $a_{i,} b_i$ for $i \in \{1, \ldots, n\}$ such that $X_i \in [a_i, b_i]$. For all $\epsilon > 0$,

$$\mathbb{P}\left(\sum_{i=1}^{n} X_i - \mathbb{E}\left[\sum_{i=1}^{n} X_i\right] \geq \epsilon\right) \leq \exp\left\{-\frac{2\epsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right\}.$$

Proof: Start by simple rewriting and application of Markow's inequality - which is legal since $S_X \equiv \sum_{i=1}^n X_i$ are composed of i.i.d. stochastic variables and thus $S_X - \texttt{constant}$ itself is a stochastic variable (here $\texttt{constant} = \mathbb{E}\left(\sum_{i=1}^n X_i\right)$

$$\begin{aligned}\mathbb{P}\left(\sum_{i=1}^n X_i - \mathbb{E}\left[\sum_{i=1}^n X_i\right] \geq \epsilon\right) &= \mathbb{P}\left(e^{\lambda(\sum_{i=1}^n X_i - \mathbb{E}[\sum_{i=1}^n X_i])} \geq e^{\lambda\epsilon}\right) \\ &\leq \frac{E\left[e^{\lambda(\sum_{i=1}^n X_i - \mathbb{E}[\sum_{i=1}^n X_i])}\right]}{e^{\lambda\epsilon}}.\end{aligned}$$

The nominator can be reduced. Firstly we use that the expectation operator is linear so that $\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i]$

$$\begin{aligned}\mathbb{E}\left[e^{\lambda(\sum_{i=1}^n X_i - \mathbb{E}[\sum_{i=1}^n X_i])}\right] &= \mathbb{E}\left[e^{\lambda(\sum_{i=1}^n X_i - \sum_{i=1}^n \mathbb{E}[X_i])}\right] \\ &= \mathbb{E}\left[e^{\lambda\sum_{i=1}^n (X_i - \mathbb{E}[X_i])}\right].\end{aligned}$$

Now using the properties of the exponential function and Hoeffdings lemma

$$\begin{aligned}\mathbb{E}\left[e^{\lambda\sum_{i=1}^n X_i - \mathbb{E}[X_i]}\right] &= \mathbb{E}\left[\prod_{i=1}^n e^{\lambda(X_i - \mathbb{E}[X_i])}\right] \\ &\overset{indep.}{=} \prod_{i=1}^n E\left[e^{\lambda(X_i - \mathbb{E}[X_i])}\right] \\ &\leq \prod_{i=1}^n e^{\lambda^2(b_i-a_i)^2/8} \\ &= e^{\lambda^2/8\sum_{i=1}^n (b_i-a_i)^2}.\end{aligned}$$

Taking stock: so far we have

$$\mathbb{P}\left(\sum_{i=1}^n X_i - \mathbb{E}\left[\sum_{i=1}^n X_i\right] \geq \epsilon\right) \leq e^{\lambda^2/8\sum_{i=1}^n (b_i-a_i)^2 - \lambda\epsilon}.$$

However we want the tightest probability bound, and therefore we minimize

the right hand side expression wrt. $\lambda > 0$.

$$\lambda^* = \arg \min_{\lambda \in \mathbb{R}_+} \; e^{\lambda^2/8 \sum_{i=1}^n (b_i - a_i)^2 - \lambda \epsilon} = \arg \min_{\lambda \in \mathbb{R}_+} \left\{ \frac{\lambda^2}{8} \sum_{i=1}^n (b_i - a_i)^2 - \lambda \epsilon \right\},$$

with necessary first order condition

$$0 = \frac{\lambda^*}{4} \sum_{i=1}^n (b_i - a_i)^2 - \epsilon \iff \lambda^* = \frac{4\epsilon}{\sum_{i=1}^n (b_i - a_i)^2}.$$

Substituting this into the stock-taking expression yields the theorem statement

$$\begin{aligned} e^{(\lambda^*)^2/8 \sum_{i=1}^n (b_i - a_i)^2 - \lambda^* \epsilon} &= \exp \left\{ \left(\frac{4\epsilon}{\sum_{i=1}^n (b_i - a_i)^2}. \right)^2 \frac{1}{8} \sum_{i=1}^n (b_i - a_i)^2 - \frac{4\epsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right\} \\ &= \exp \left\{ \frac{2\epsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} - \frac{4\epsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right\} \\ &= \exp \left\{ -\frac{2\epsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right\}. \end{aligned}$$

Taken all together we have obtained Hoeffding Inequality:

$$\mathbb{P} \left(\sum_{i=1}^n X_i - \mathbb{E} \left[\sum_{i=1}^n X_i \right] \geq \epsilon \right) \leq \exp \left\{ -\frac{2\epsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right\}.$$

2 Generalization bounds: Finite and Countably Infinity Hypothesis Classes

2.1 Single hypothesis

2.1.1 Setting and risk definitions

Assume the loss function $\ell(h(X), Y)$ is bounded on $[0, 1]$. This is important for the specific calculations but the general insights are applicable to any loss function bounds $[a, b]$.

True risk: The quality of any hypothesis h is measured by its (true) risk

$$L(h) \equiv \mathbb{E}_{(X,Y)\sim p}\left[\ell(h(X), Y)\right],$$

where p is the *unknown* distribution over which the world has generated (X, Y) - i.e. pairs of features from the input space $\mathcal{X}$ and outcomes from the label space $\mathcal{Y}$.

Empirical risk: Using a validation sample

$$S = \{(X_1, Y_1), \ldots, (X_N, Y_N)\}, \quad (X_i, Y_i) \overset{iid}{\sim} p,$$

we calculate the empirical risk defined by

$$\hat{L}(h, S) \equiv \frac{1}{N}\sum_{n=1}^{N} \ell(h(X_n), Y_n).$$

2.1.2 Theoretical properties of empirical risk

Unbiased: Since the validation samples are independently and identically distributed from the distribution p we have

$$\begin{aligned}
\mathbb{E}_{S\sim p^N}\left[\hat{L}(h, S)\right] &= \mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N} \ell(h(X_n), Y_n)\right] \\
&= \frac{1}{N}\sum_{n=1}^{N} \mathbb{E}_{(X_n,Y_n)\sim p}\left[\ell(h(X_n), Y_n)\right] \\
&= \frac{1}{N}\sum_{n=1}^{N} \mathbb{E}_{(X_1,Y_1)\sim p}\left[\ell(h(X_1), Y_1)\right] \\
&= \mathbb{E}_{(X_1,Y_1)\sim p}\left[\ell(h(X_1), Y_1)\right] \\
&= L(h).
\end{aligned}$$

However the above only holds because h is independent of S. If not we generally have

$$\mathbb{E}_{S\sim p^N}\left[\hat{L}\left(\hat{h}_S, S\right)\right] \neq \mathbb{E}_{S\sim p^N}\left[L\left(\hat{h}_S\right)\right]$$

What can be said about the distribution of $\hat{L}(h,S)$? Formally what is δ here

$$\mathbb{P}\left(L(h) - \hat{L}(h,S) \geq \epsilon\right) \leq \delta.$$

I.e. with what tightness can we bound the probability that the true risk L is $\epsilon > 0$ bigger than $\hat{L}$. For example we might want a confindence of $\delta = 0.05$ that is w.p. less than 0.05 the true loss L is $\epsilon = f(\delta)$ - that depends on δ units bigger than $\hat{L}$.

Applying Hoeffdings bound yields

$$\begin{aligned}
\mathbb{P}\left(L(h) - \hat{L}(h,S) \geq \epsilon\right) &= \mathbb{P}\left(\underbrace{\mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\ell(h(X_n),Y_n)\right]}_{=\mathbb{E}[\ell(h(X_1),Y_1)]=L(h)} - \frac{1}{N}\sum_{n=1}^{N}\ell(h(X_n),Y_n) \geq \epsilon\right) \\
&= \mathbb{P}\left(\mathbb{E}\left[\sum_{n=1}^{N}\ell(h(X_n),Y_n)\right] - \sum_{n=1}^{N}\ell(h(X_n),Y_n) \geq N\epsilon\right) \\
&\leq \exp\left\{-\frac{2(N\epsilon)^2}{\sum_{n=1}^{N}(b_n-a_n)^2}\right\} \\
&= \exp\left\{-2N\epsilon^2\right\}.
\end{aligned}$$

Defining the relation $\delta = \exp\left\{-2N\epsilon^2\right\}$ yields $\epsilon = \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$. Thus

$$\mathbb{P}\left(L(h) \geq \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) \leq \delta,$$

and taking the complement

$$\mathbb{P}\left(L(h) < \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) = 1 - \mathbb{P}\left(L(h) \geq \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) \geq 1 - \delta.$$

2.1.3 *Sidenote: Understanding* $\epsilon = \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$

Earlier we obtained

$$\mathbb{P}\left(L(h) - \hat{L}(h,S) \geq \epsilon\right) \leq \exp\left\{-2N\epsilon^2\right\}.$$

we can think if this inequality as asking: *If you set an error-margin* $\epsilon > 0$ *then I tell you how confident I am that an error happens (i.e.* $L - \hat{L} \geq \epsilon$*).* If we instead ask the question: *I want the probability that an error happens* $(i.e. L - \hat{L} \geq \epsilon)$ *to be* $\delta \cdot 100\%$ *how large does the error-margin* ϵ *have to be?* The latter case is answered by relabeling the right hand side as $\delta = \exp\left\{-2N\epsilon^2\right\}$ and then backing out what ϵ must be: $\epsilon = \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$.

The complement flips the question:

$$\mathbb{P}\left(L(h) < \hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}\right) \geq 1 - \delta,$$

and asks: The probability that L is acceptable (i.e. smaller than $\hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$) is at least $1 - \delta$. Or: I am $(1-\delta)\cdot 100\%$ certain that L is smaller than $\hat{L}(h,S) + \sqrt{\frac{1}{2N}\ln\left(\frac{1}{\delta}\right)}$.

2.2 Finite Hypothesis class

Unlike before we have now trained $h_1, \ldots, h_M$ hypotheses as such

$$
\begin{aligned}
h_1 &\longrightarrow \hat{L}(h_1, S) = \frac{1}{N}\sum_{n=1}^{N} \ell(h_1(X_n), Y_n) \\
h_2 &\longrightarrow \hat{L}(h_2, S) = \frac{1}{N}\sum_{n=1}^{N} \ell(h_2(X_n), Y_n) \\
&\vdots \\
h_M &\longrightarrow \hat{L}(h_M, S) = \frac{1}{N}\sum_{n=1}^{N} \ell(h_M(X_n), Y_n),
\end{aligned}
$$

and chosen $\hat{h_S^*}$ with empirical risk minimization

$$
\begin{aligned}
\hat{h_S^*} &= \arg\min_{h \in \mathcal{H}} \hat{L}(h, S) \\
&= \arg\min_{h \in \mathcal{H}} \left\{ \frac{1}{N}\sum_{n=1}^{N} \ell(h(X_n), Y_n) \right\},
\end{aligned}
$$

where S is a validation sample (i.e. not used for training) with elements $(X_n, Y_n) \overset{iid}{\sim} p$.

2.2.1 *Hoeffdings bound does not apply anymore!*

The question is as follows: *How bad can we expect our model to perform?* This is measured in terms of (true) risk

$$
\mathbb{E}_{(X,Y)\sim p}\left[\ell\left(\hat{h_S^*}(X), Y\right)\right],
$$

where p is the probability distribution of the stochastic vector (X, Y). We would ideally just plug and play with Hoeffding's inequality. However this requires that

$$
L\left(\hat{h_S^*}\right) = \mathbb{E}_S\left[\hat{L}\left(\hat{h_S^*}, S\right)\right],
$$

since

$$\mathbb{P}\left(L\left(\hat{h}^*_S\right) - \hat{L}\left(\hat{h}^*_S, S\right) \geq \epsilon\right) = \mathbb{P}\left(L\left(\hat{h}^*_S\right) - \frac{1}{N}\sum_{n=1}^{N} \ell\left(\hat{h}^*_S(X_n), Y_n\right) \geq \epsilon\right)$$
$$= \mathbb{P}\left(NL\left(\hat{h}^*_S\right) - \sum_{n=1}^{N} \ell\left(\hat{h}^*_S(X_n), Y_n\right) \geq N\epsilon\right)$$

For Hoeffdings bound to apply we would therefore need that[1]

$$NL\left(\hat{h}^*_S\right) = \mathbb{E}_{S\sim p^N}\left[\sum_{n=1}^{N} \ell\left(\hat{h}^*_S(X_n), Y_n\right)\right]$$

which implies

$$L\left(\hat{h}^*_S\right) = \mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N} \ell\left(\hat{h}^*_S(X_n), Y_n\right)\right] = \mathbb{E}_{S\sim p^N}\left[\hat{L}\left(\hat{h}^*_S, S\right)\right].$$

This however does not generally hold since

$$\mathbb{E}_{S\sim p^N}\left[\hat{L}\left(\hat{h}^*_S, S\right)\right] = \mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N} \ell\left(\hat{h}^*_S(X_n), Y_n\right)\right]$$
$$\neq \mathbb{E}_{(X_1,Y_1)\sim p}\left[\frac{1}{N}\sum_{n=1}^{N} \ell\left(\hat{h}^*_S(X_1), Y_1\right)\right],$$

since $\hat{h^*}_S$ generally depends on the whole sample $S = \{(X_1, Y_1), \ldots, (X_N, Y_N)\}$. Therefore the stochastic variables $\ell\left(\hat{h^*}_S\left(X_n\right), Y_n\right)$ are not independent (because $\hat{h^*}_S$ was selected using $S_{-(X_n,Y_n)}$) and therefore Hoeffding's inequality does not apply.

[1] To see this simply look at the inequality

$$\mathbb{P}\left(\mathbb{E}\left[\sum_{n=1}^{N} X_n\right] - \sum_{n=1}^{N} X_n \geq \epsilon\right) \leq \exp\left\{-\frac{2\epsilon^2}{\sum_{n=1}^{N}(b_n - a_n)^2}\right\},$$

from where we see that the left term needs to be $\mathbb{E}\left[\sum_{n=1}^{N} X_n\right]$ i.e. the expectation of the right term $\sum_{n=1}^{N} X_n$.

The solution: Therefore we somehow need to break dependence of the hypothesis $\hat{h^*}_S$ on the sample:

$$\begin{aligned}
\mathbb{P}\left(L\left(\hat{h^*_S}\right) - \hat{L}\left(\hat{h^*_S}, S\right) \geq \epsilon\right) &\leq \mathbb{P}\left(L\left(h_1\right) - \hat{L}\left(h_1, S\right) \geq \epsilon \text{ or } \cdots \text{ or } L\left(h_M\right) - \hat{L}\left(h_M, S\right) \geq \epsilon\right) \\
&= \mathbb{P}\left(\exists h \in \mathcal{H} : L\left(h\right) - \hat{L}\left(h, S\right) \geq \epsilon\right) \\
&\leq \sum_{j=1}^{M} \mathbb{P}\left(L\left(h_j\right) - \hat{L}\left(h_j, S\right) \geq \epsilon\right) \\
&= \sum_{j=1}^{M} \mathbb{P}\left(N \underbrace{L\left(h_j\right)}_{\equiv \mathbb{E}_{(X,Y)\sim p}\left[\ell\left(h_j(X), Y\right)\right]} - \sum_{n=1}^{N} \ell\left(h_j(X_n), Y_n\right) \geq N\epsilon\right) \\
&= \sum_{j=1}^{M} \mathbb{P}\left(N\mathbb{E}_{S\sim p^N}\left[\frac{1}{N}\sum_{n=1}^{N} \ell\left(h_j(X), Y\right)\right] - \sum_{n=1}^{N} \ell\left(h_j(X_n), Y_n\right) \geq N\epsilon\right) \\
&\leq M \exp\left\{-\frac{2(N\epsilon)^2}{\sum_{n=1}^{N}(b_n - a_n)^2}\right\} \\
&= M \exp\left\{-2N\epsilon^2\right\},
\end{aligned}$$

The first inequality holds simply because $\hat{h^*_S} \in \{h_1, h_2, \ldots, h_M\}$. The second inequality holds by the union bound:

$$\mathbb{P}\left(A \cup B\right) = P(A) + P(B) - \mathbb{P}\left(A \cap B\right) \leq P(A) + P(B).$$

From this we obtain

$$\delta = M \exp\left\{-2N\epsilon^2\right\} \iff \epsilon = \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)},$$

and further

$$\begin{aligned}
\mathbb{P}\left(L\left(\hat{h^*_S}\right) \geq \hat{L}\left(\hat{h^*_S}, S\right) + \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)}\right) &\leq \delta \\
\mathbb{P}\left(L\left(\hat{h^*_S}\right) < \hat{L}\left(\hat{h^*_S}, S\right) + \sqrt{\frac{1}{2N} \ln\left(\frac{M}{\delta}\right)}\right) &\geq 1 - \delta.
\end{aligned}$$

This is the result: From a finite predetermined hypothesis class with cardinality $|\mathcal{H}| = M$ using the Empirical Risk Minimizer (ERM) - or any selection over h! Since:

$$\mathbb{P}\left(\exists h \in \mathcal{H} : L\left(h\right) \geq \hat{L}\left(h, S\right) + \sqrt{\frac{1}{2N}\ln\left(\frac{M}{\delta}\right)}\right) \leq \delta,$$

and in particular for the ERM hypothesis $\hat{h}_S^*$

$$\mathbb{P}\left(L\left(\hat{h}_S^*\right) \geq \hat{L}\left(\hat{h}_S^*, S\right) + \sqrt{\frac{1}{2N}\ln\left(\frac{M}{\delta}\right)}\right) \leq \delta.$$

2.3 Lower Bound

In general there is a lower bound on how well bounding inequalities can work. Concretely we use Hoeffding's inequality, but could this be improved? And how much? To answer the first: yes, it has been improved. For the second question: we can show that there is a limit to how much bounding can happen in general. We call this a lower bound which answers the question: In general what is the lowest upper bound we can impose on an event? or: It tells us that we can never be *this* certain (i.e. δ cannot be this low) using bounding techniques.

In short the lectures construct an example for a specific hypothesis space $\mathcal{H}$ where

$$\mathbb{P}\left(\exists h \in \mathcal{H} : L\left(h\right) \geq \hat{L}\left(h, S\right) + \frac{1}{8}\right) \geq \frac{1}{8}.$$

2.4 Approximation and Estimation error

Approximation error comes from restricting $\mathcal{H}$ so that the Bayes optimal hypothetical choice is omittet (i.e. the h that would generate the lowest $L(h)$ is not in $\mathcal{H}$).

Estimation defines the variance in selecting the best hypothesis in class $\mathcal{H}$: there is an $h^* = \arg\min_{h\in\mathcal{H}} L(h)$ but using data we are not sure to find it. we most likely find $L(\hat{h}) < L(h^*)$. This is the estimation error.

II | Supervised Learning

3 Linear regression

3.1 Linear Algebra prerequisites

3.1.1 Singular Value Decomposition

Statement: For any real matrix $X \in \mathbb{R}^{N \times d}$ we can write

$$X = U\Gamma V',$$

where U and V are semi-orthogonal and Γ is a diagonal matrix with non-negative entries. The dimensions of all matrices depend on $r = rank(X)$ see proof.

Proof: Consider the symmetric matrix $B = X'X \in \mathbb{R}^{n \times n}$ (which can be shown in one line: $(X'X)' = X'X$) that by the spectral theorem B has n orthonormal eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$. Furthermore this implies that the eigenvalues are non-negative:

$$\begin{aligned}
\lambda_i &= \lambda_i \mathbf{v}_i' \mathbf{v}_i \\
&= \mathbf{v}_i' \lambda_i \mathbf{v}_i \\
&= \mathbf{v}_i' (X'X) \mathbf{v}_i \\
&= (X\mathbf{v}_i)'(X\mathbf{v}_i) \\
&= ||X\mathbf{v}_i||_2^2 \geq 0.
\end{aligned}$$

Now define the singular values $\sigma_i \equiv \sqrt{\lambda_i}$ and define $\mathbf{u}_i \equiv X\mathbf{v}_i/\sigma_i$. The latter can be shown to be orthonormal for the collection $i = 1, \ldots, r$ where $r = rank(X)$.

To verify orthonormality, note that for $i, j \leq r$,

$$\begin{aligned}
\mathbf{u}_i' \mathbf{u}_j &= \frac{\mathbf{v}_i' X' X \mathbf{v}_j}{\sigma_i \sigma_j} \\
&= \frac{\lambda_j}{\sigma_i \sigma_j} \mathbf{v}_i' \mathbf{v}_j \\
&= \frac{\sigma_j}{\sigma_i} \delta_{ij} = \delta_{ij}.
\end{aligned}$$

We can complete these r vectors to an orthonormal basis of $\mathbb{R}^N$, forming the orthogonal matrix $U = [\mathbf{u}_1, \ldots, \mathbf{u}_N]$. Similarly, let $V = [\mathbf{v}_1, \ldots, \mathbf{v}_d]$.

By construction, $X\mathbf{v}_i = \sigma_i \mathbf{u}_i$ for $i \leq r$. For $i > r$, we have $\lambda_i = 0$, implying $\|X\mathbf{v}_i\|_2^2 = 0$ and hence $X\mathbf{v}_i = 0$.

Consequently,

$$XV = U\Gamma,$$

where $\Gamma \in \mathbb{R}^{N \times d}$ is rectangular diagonal with entries $\sigma_1, \ldots, \sigma_r > 0$ and zeros elsewhere. Since $VV' = I_d$,

$$X = U\Gamma V'.$$

This holds regardless of whether $N > d$ or $N < d$.

3.1.2 The Spectral Theorem

Statement: Any real valued symmetrix matrix $A \in \mathbb{R}^{n \times n}$ can be written as

$$A = Q\Lambda Q',$$

where $Q = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} \in \mathbb{R}^{n \times n}$ holds n orthogonal real-valued eigenvectors and $\Lambda = diag(\lambda_1, \cdots, \lambda_n)$ holds their respective eigenvalues.

Proof: Firstly we show that any real valued symmetric (i.e. $A = A'$) matrix $A \in \mathbb{R}^{n \times n}$ has at least one real-valued eigenvector $\mathbf{v}_1$. Then we show that A preserves the subspace orthogonal to $\mathbf{v}_1$ that is $\mathbf{v}_1' A\mathbf{x} = 0$ iff $\mathbf{v}_1'\mathbf{x} = 0$ and using this fact we iteratively contruct n real-valued orthogonal eigenvectors.

We want to maximise the following (subject to the unit circle)

$$\max_{\mathbf{x}'\mathbf{x}=1} \mathbf{x}' A\mathbf{x},$$

with the lagrangian

$$\mathcal{L}(\mathbf{x}) = \mathbf{x}' A\mathbf{x} - \lambda \left(\mathbf{x}'\mathbf{x} - 1\right),$$

and its first order conditions

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}}|_{\mathbf{x}=\mathbf{x}^*} = (A + A')\mathbf{x}^* - 2\lambda\mathbf{x}^* = 2A\mathbf{x}^* - 2\lambda\mathbf{x}^* = 0$$
$$A\mathbf{x}^* = \lambda\mathbf{x}^*,$$

where the second equality uses symmetry $A = A'$. Thus we have shown that there exists a vector $\mathbf{v}_1 = \mathbf{x}^*$ with the associated eigenvalue λ_1. From here we continue by finding the next eigenvector. We simply solve the same maximisation problem with the added restriction $\mathbf{v}_1'\mathbf{x} = 0$:

$$\max_{\mathbf{x}'\mathbf{x}=1, \mathbf{v}_1'\mathbf{x}=0} \mathbf{x}'A\mathbf{x},$$

with the lagrangian

$$\mathcal{L}(\mathbf{x}) = \mathbf{x}'A\mathbf{x} - \lambda\left(\mathbf{x}'\mathbf{x} - 1\right) - \mu\left(\mathbf{v}_1'\mathbf{x}\right),$$

and the first order condition

$$A\mathbf{x}^* = \lambda\mathbf{x}^* + \mu\mathbf{v}_1 = \lambda\mathbf{x}^*,$$

where the last inequality (i.e. $\mu = 0$) follows from symmetry of A.[2] Therefore we have found another eigenvector which we name $\mathbf{v}_2$ and its associated eigenvalue λ_2. Keep going like this and find all n eigenvectors to obtain

$$A\mathbf{v}_i = \lambda_i\mathbf{v}_i,\ i = 1, \ldots, n,$$

[2]To see this let $\mathbf{v}_1'\mathbf{w} = 0$ then because of symmetry

$$\mathbf{v}_1'A\mathbf{w} = (A\mathbf{v}_1)'\mathbf{w} = (\lambda\mathbf{v}_1)'\mathbf{w} = \lambda\mathbf{v}_1'\mathbf{w} = 0$$

and therefore

$$\underbrace{\mathbf{v}_1'A\mathbf{x}^*}_{=0} = \lambda\underbrace{\mathbf{v}_1'\mathbf{x}^*}_{=0} + \mu\underbrace{\mathbf{v}_1'\mathbf{v}_1}_{=1} \iff \mu = 0.$$

which allows us to write

$$AQ = Q\Lambda,$$

where $Q = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} \in \mathbb{R}^{n \times n}$ and $\Lambda = diag(\lambda_1, \cdots, \lambda_n)$. Now remember that the eigenvectors are constructed iterately where the jth eigenvector is orthogonal to the preceeding $j-1$ eigenvectors.

$$\mathbf{v}_i' \mathbf{v}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases},$$

which can be collected into a matrix $Q = \begin{bmatrix} \mathbf{v}_1' & \mathbf{v}_2' & \cdots & \mathbf{v}_n' \end{bmatrix}$ such that

$$QQ' = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \\ \mathbf{v}_n \end{bmatrix} \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} = I_n.$$

Therefore we have

$$AQ = Q\Lambda \iff A = Q\Lambda Q'$$

thus completing the proof.

3.2 Overidentified case: $N > rank(X)$

Let the design matrix be defined as

$$X = \begin{bmatrix} \phi(\mathbf{x}_1)' & 1 \\ \vdots & \vdots \\ \phi(\mathbf{x}_n)' & 1 \end{bmatrix} \in \mathbb{R}^{N \times d+1},$$

where $\phi(\mathbf{x}_i) \in \mathbb{R}^d$ is observation $i = 1, \ldots, N$ which is possibly transformed / constructed from the raw data. That is the raw data for person / observation i,

$\mathbf{x}_i$ is used to extract the features $\phi(\mathbf{x}_i) \in \mathbb{R}^d$. For example we could have

$$\phi(\begin{bmatrix} age_i & height_i \end{bmatrix}') = \begin{bmatrix} age_i & age_i^2 & height_i \end{bmatrix}',$$

so that the dimensionality of the feature space is $d = 3$ and larger than the underlying raw datasets 2 dimensions.

Problem: For $N > d + 1$ and no linearly dependent columns in X then $X'X$ has full rank and the following minimisation problem (using the MSE as loss function) has exactely one solution

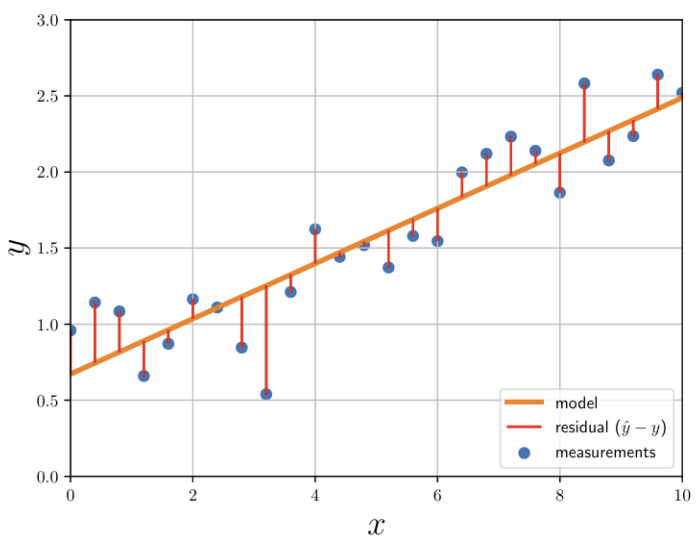
$$\begin{aligned} \hat{\mathbf{w}} =& \arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} \frac{1}{N} \sum_{i=1}^{N} (y_i - \mathbf{w}' \tilde{\phi}(\mathbf{x}_i))^2 \\ =& \arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} \frac{1}{N} (\mathbf{y} - X\mathbf{w})' (\mathbf{y} - X\mathbf{w}), \end{aligned}$$

where $\tilde{\phi}(\mathbf{x}_i) = \begin{bmatrix} \phi(\mathbf{x}_i) & 1 \end{bmatrix}' \in \mathbb{R}^{d+1}$ describes the possible transformation of the raw underlying data $\mathbf{x}_i$.

Solution: The first order condition of the problem is

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \frac{2}{N} X'(\mathbf{y} - X\mathbf{w}) = 0 \\ \mathbf{w} &= (X'X)^{-1} X'\mathbf{y}, \end{aligned}$$

which yields the well known OLS estimator. Here the error is visualized:



3.2.1 *OLS is equivalent with Gaussian MLE*

Assume $y_i = f_{\mathbf{w}}(\mathbf{x}_i) + \varepsilon_i$, where $\varepsilon_i \overset{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2)$. Then the likelihood is

$$L(\mathbf{w}) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \mathbf{w}) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2}{2\sigma^2}\right).$$

Taking the negative log-likelihood gives

$$-\log L(\mathbf{w}) = \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2.$$

Hence, for fixed σ^2, maximum likelihood estimation is equivalent to minimizing the mean squared error:

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} L(\mathbf{w}) = \arg\min_{\mathbf{w}} \frac{1}{N}\sum_{i=1}^{N}(y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2.$$

3.3 Underidentified case: $N < rank(X)$

When $N < rank(X)$ the system of equations $\mathbf{w} = (X'X)^{-1}X'\mathbf{y}$ has infinitely many solutions. Therefore we need a way to choose a good $\mathbf{w}$ amongs the many

solutions that satisfy $\mathbf{w}_i' \mathbf{x}_i = y_i,\ i = 1, \ldots, N$.

3.3.1 Minimum-norm solution

Problem: One sensible way to estimate the model is to choose the weighting vector $\mathbf{w}$ with the smallest euclidian norm where the model still fits the training data perfectly:

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} ||\mathbf{w}||_2^2 \text{ s.t. } \forall i : \mathbf{w}_i' \mathbf{x}_i = y_i$$

Solution: We solve the constrained optimization problem using that the linear constraint can be written as $\mathbf{w}_i' \mathbf{x}_i = y_i \iff X\mathbf{w} = \mathbf{y}$.

Setting up the lagrangian

$$\begin{aligned} \mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}) &= \mathbf{w}^\top w - \sum_{n=1}^{N} \lambda_n \left(\mathbf{w}_n' \mathbf{x}_n - y_n \right) \\ &= \mathbf{w}^\top \mathbf{w} - \boldsymbol{\lambda}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}), \end{aligned}$$

First-order condition with respect to $\mathbf{w}$ gives

$$\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{w} - X^\top \boldsymbol{\lambda} = 0 \quad \Longrightarrow \quad \mathbf{w} = \frac{1}{2} X^\top \boldsymbol{\lambda}.$$

Substituting into the constraint yields

$$X\mathbf{w} = \mathbf{y} \quad \Longrightarrow \quad \frac{1}{2} X X^\top \boldsymbol{\lambda} = \mathbf{y} \quad \Longrightarrow \quad \boldsymbol{\lambda} = 2(XX^\top)^{-1} \mathbf{y}.$$

Hence, the minimum-norm solution is

$$\hat{\mathbf{w}} = \underbrace{X^\top (XX^\top)^{-1}}_{= X^\dagger} \mathbf{y}.$$

3.3.2 *Singular Value Decomposition and the Pseudoinverse*

Any matrix $X \in \mathbb{R}^{N \times d}$ admits a singular value decomposition (SVD)

$$X = U\Gamma V^\top,$$

where, for $r = \min\{N, d\}$, $U \in \mathbb{R}^{N \times r}$ and $V \in \mathbb{R}^{d \times r}$ are semi-orthogonal, and $\Gamma = \operatorname{diag}(\sigma_1, \ldots, \sigma_r)$ contains the singular values $\sigma_1 \geq \cdots \geq \sigma_r \geq 0$.

Ordinary Least Squares ($N \geq d$). Assuming full column rank, the OLS estimator is

$$\begin{aligned}\hat{\mathbf{w}}_{\text{OLS}} &= (X^\top X)^{-1} X^\top \mathbf{y} \\ &= (V\Gamma^2 V^\top)^{-1} V\Gamma U^\top \mathbf{y} \\ &= V\Gamma^{-1} U^\top \mathbf{y}.\end{aligned}$$

Minimum-Norm Interpolation ($N \leq d$). Assuming full row rank, the minimum-norm solution satisfying $X\mathbf{w} = \mathbf{y}$ is

$$\begin{aligned}\hat{\mathbf{w}}_{\text{MN}} &= X^\top (XX^\top)^{-1} \mathbf{y} \\ &= V\Gamma U^\top (U\Gamma^2 U^\top)^{-1} \mathbf{y} \\ &= V\Gamma^{-1} U^\top \mathbf{y}.\end{aligned}$$

Thus, both estimators can be expressed using the **Moore-Penrose pseudoinverse**:

$$\hat{\mathbf{w}} = X^\dagger \mathbf{y}, \qquad X^\dagger = V\Gamma^\dagger U^\top,$$

where $\Gamma^\dagger$ is diagonal with entries $1/\sigma_i$ if $\sigma_i > 0$ and 0 otherwise. This also holds for rank-deficient matrices, in which case the pseudoinverse selects the minimum-norm least-squares solution.

Why use SVD? Computing $(X^\top X)^{-1}$ directly can be numerically unstable because forming $X^\top X$ squares the condition number:

$$\kappa(X^\top X) = \kappa(X)^2.$$

SVD avoids this squaring and handles rank deficiency by inverting only nonzero singular values. However, very small singular values can still amplify numerical errors, motivating regularization or truncation.

3.3.3 Ridge regression (Two-norm regularization)

Problem: Add regularisation term $\lambda||\mathbf{w}||_2^2$ with $\lambda > 0$ and the problem of ridge regression becomes

$$\begin{aligned}\hat{\mathbf{w}} &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}\sum_{i=1}^{N}(y_i - \mathbf{w}'\tilde{\boldsymbol{\phi}}(\mathbf{x}_i))^2 + \lambda||\mathbf{w}||_2^2 \\ &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}(\mathbf{y} - X\mathbf{w})'(\mathbf{y} - X\mathbf{w}) + \lambda\mathbf{w}'\mathbf{w},\end{aligned}$$

Solution: The first order condition (of the lagrangian) are

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{w}}|_{\mathbf{w}=\hat{\mathbf{w}}} &= \frac{2}{N}\left(X'(\mathbf{y} - X\hat{\mathbf{w}}) + \lambda\hat{\mathbf{w}}\right) = 0 \\ \hat{\mathbf{w}} &= (X'X + \lambda I_{d+1})X'\mathbf{y}.\end{aligned}$$

Note that for $\lambda \to 0$ the solution approaches that of OLS and for $\lambda \to \infty$ the solution goes to 0 $\hat{\mathbf{w}} \to \mathbf{0}$. Intuitively this is because weight-elements other than 0 are penalised a lot.

Choosing regularization paramter: λ can be chosen by cross validation (choosing the λ^* minimzing CV risk).

3.3.4 Equivalence theorem

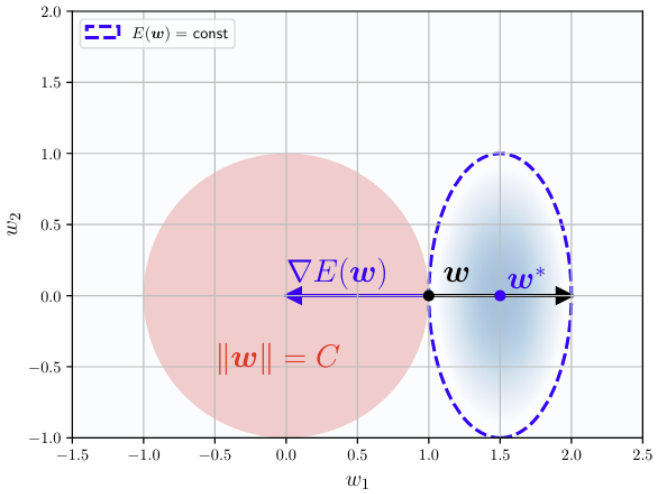
Ridge regression can be seen as solving

$$\arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}\sum_{i=1}^{N}(y_i - \tilde{\boldsymbol{\phi}}(\mathbf{x}_i))^2 \text{ s.t. } ||\mathbf{w}||_2 \leq C,$$

becuase this is equivalent to the ridge regression problem:

$$\arg \min_{\mathbf{w} \in \mathbb{R}^{d+1}} \frac{1}{N} (\mathbf{y} - X\mathbf{w})' (\mathbf{y} - X\mathbf{w}) + \lambda \mathbf{w}'\mathbf{w},$$

for some $\lambda = f(C)$ depending on C. This is visualized here



This intepretation is useful because of what the restricted hypothesis space (where $||\mathbf{w}||_2 \leq C$) tries to implement for the affine linear hypothesis $\tilde{\boldsymbol{\phi}}(\mathbf{x}_i)$). Formally let $\tilde{\phi} : \mathbb{R}^{d_0} \to \mathbb{R}^{d+1}$ (where d_0 is the dimensions of the raw underlying data which the featurespace $\mathbb{R}^{d+1}$ is created from). be continuous and Lipschits with constant α meaning:

$$||\tilde{\boldsymbol{\phi}}(\mathbf{x}_1) - \tilde{\boldsymbol{\phi}}(\mathbf{x}_2)||_2 \leq \alpha ||\mathbf{x}_1 - \mathbf{x}_2||_2,$$

for $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^{d_0}$. Then the functions $h : \mathbb{R}^{d_0} \to \mathbb{R}^1$ in

$$\mathcal{H}_C = \left\{ h(\mathbf{x}) = \mathbf{w}'\tilde{\boldsymbol{\phi}}(\mathbf{x}) \mid \mathbf{w} \in \mathbb{R}^{d+1} \text{ and } ||\mathbf{w}||_2 \leq C \right\},$$

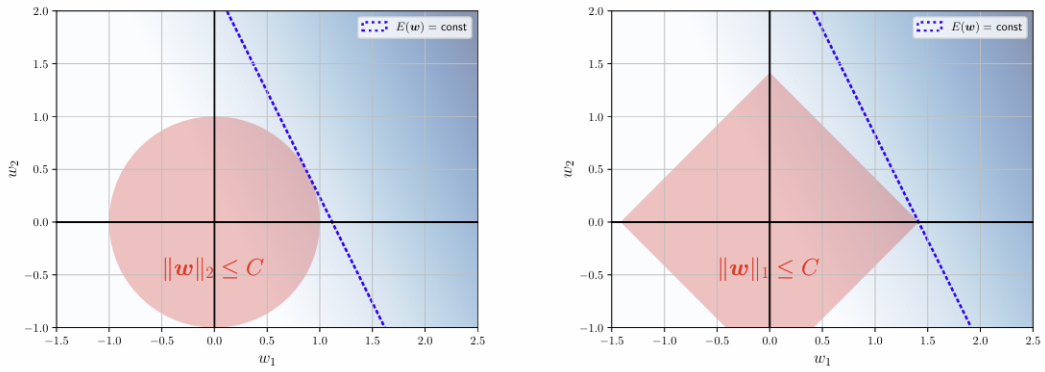
will be Lipschitz with constant $C \cdot \alpha$. In other words C limits how fast the prediction rule / hypothesis h can change.

3.3.5 *Lasso regression (One-norm regularization)*

Much the same as ridge regression but where we add the l1-norm regularization (to reduce the number of active dimensions - i.e. increase number of weights equal to 0 in $\mathbf{w}$)

$$\begin{aligned}\hat{\mathbf{w}} &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}\sum_{i=1}^{N}(y_i - \mathbf{w}'\tilde{\boldsymbol{\phi}}(\mathbf{x}_i))^2 + \lambda||\mathbf{w}||_1 \\ &= \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \frac{1}{N}(\mathbf{y} - X\mathbf{w})'(\mathbf{y} - X\mathbf{w}) + \lambda\sum_{j=1}^{d}|w_j|\end{aligned}$$

The sparcity effect is visualized here



3.4 Comment on double descent

Sometimes when letting the number of parameters grow $d >> N$ - while including a regularization term like $\lambda||\mathbf{w}||_2^2$, $\lambda > 0$ then at some point testing error can start diminishing rather than keep increasing. This can even result in lower empirical test loss than models with $d < N$.

The name comes from the stylized fact that test loss decreases when adding model dimensions d and d is low. At some point the model overfits and testing empirical risk increases. And then the magic can enter for $d >> N$ where the test loss again decreases!

4 Linear Classification

The task of linear classification is to predict the correct class $\mathcal{Y} \in \{\mathcal{C}_1, \ldots, \mathcal{C}_m\}$, $2 \leq m$ from the input $\mathbf{x}_i \in \mathcal{X}$. One approach is to learn the discriminant functions $\delta_k : \mathcal{X} \to \mathbb{R}$, $k = 1, \ldots, m$

$$\hat{y}_i = h(\mathbf{x}_i) = \arg \max_{k \in \{1,\ldots,m\}} \delta_k(\mathbf{x}_i)$$

We specify affine linear discriminant functions

$$\delta(\mathbf{x}_i) = \mathbf{w}' \boldsymbol{\phi}(\tilde{\mathbf{x}}_i),$$

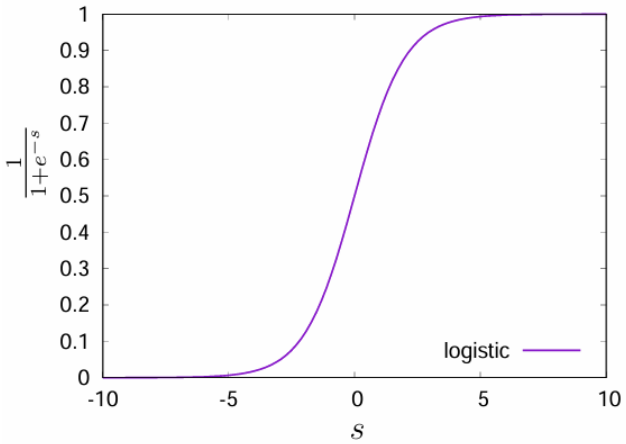
where $\mathbf{x}_i = \boldsymbol{\phi}(\tilde{\mathbf{x}}_i)$ is the features always augmentet to hold a 1 (so that the model is affine linear - just like in linear regression) and can also hold many other features created from the raw data $\tilde{\mathbf{x}}_i$.

4.1 Logistic regression

4.1.1 Binary case

Let $\mathcal{Y} \in \{1, -1\}$ and let $\theta : \mathbb{R} \to [0, 1]$ be the sigmoid function defined by

$$\theta(s) = \frac{1}{1 + \exp\{s\}}.$$



Deriving the binary sigmoid: The sigmoid encodes $\mathbf{w}'\mathbf{x}$ as the log ratio of probabilities. i.e. on log scale how relatively probable $Y = 1$ is compared to $Y = -1$.

$$\begin{aligned}
\mathbf{w}'\mathbf{x} &= \ln \frac{\mathbb{P}(Y = 1 \mid X = \mathbf{x})}{\mathbb{P}(Y = -1 \mid X = \mathbf{x})} \\
\exp\{\mathbf{w}'\mathbf{x}\} &= \frac{\mathbb{P}(Y = 1 \mid X = \mathbf{x})}{1 - \mathbb{P}(Y = 1 \mid X = \mathbf{x})} \\
(1 - \mathbb{P}(Y = 1 \mid X = \mathbf{x}))\exp\{\mathbf{w}'\mathbf{x}\} &= \mathbb{P}(Y = 1 \mid X = \mathbf{x}) \\
\mathbb{P}(Y = 1 \mid X = \mathbf{x}) &= \frac{\exp\{\mathbf{w}'\mathbf{x}\}}{1 + \exp\{\mathbf{w}'\mathbf{x}\}} \\
&= \frac{1}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} \equiv \theta(\mathbf{w}'\mathbf{x}).
\end{aligned}$$

Properties of sigmoid in the binary case: In the binary case we have

$$\theta(y\mathbf{w}'\mathbf{x}) = \begin{cases} \theta(\mathbf{w}'\mathbf{x}) & \text{if } y = 1 \\ 1 - \theta(\mathbf{w}'\mathbf{x}) & \text{if } y = -1 \end{cases}.$$

This is easy to show:

$$\begin{aligned}
1 - \theta(\mathbf{w}'\mathbf{x}) &= \frac{1 + \exp\{-\mathbf{w}'\mathbf{x}\}}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} - \frac{1}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} \\
&= \frac{\exp\{-\mathbf{w}'\mathbf{x}\}}{1 + \exp\{-\mathbf{w}'\mathbf{x}\}} \\
&= \frac{1}{1 + \exp\{\mathbf{w}'\mathbf{x}\}} \\
&= \theta(-\mathbf{w}'\mathbf{x}) \\
&= \theta(y\mathbf{w}'\mathbf{x}),\ y = -1
\end{aligned}$$

Log likelihood optimization: Therefore the negative log likelihood function

becomes

$$\begin{aligned}
-\ln L(\mathbf{w}) &= -\ln \prod_{n=1}^{N} \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}\right) \\
&= -\sum_{n=1}^{N} \ln \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}\right) \\
&= -\sum_{n=1}^{N} \ln \frac{1}{1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}} \\
&= \sum_{n=1}^{N} \ln[1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}],
\end{aligned}$$

which when divided by N is the objective function of the unconstrained minimisation problem

$$\hat{\mathbf{w}}_{MLE} = \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} \underbrace{-\frac{1}{N}\sum_{n=1}^{N} \ln[1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}]}_{Q(\mathbf{w})},$$

with first order condition

$$\begin{aligned}
\nabla_{\mathbf{w}} Q(\mathbf{w}) &= \frac{1}{N}\sum_{n=1}^{N} \frac{1}{1+\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}} \left(-\exp\{-y_n \mathbf{w}'\mathbf{x}_n\}\right)(-y_n \mathbf{x}_n) \\
&= \frac{1}{N}\sum_{n=1}^{N} \frac{y_n \mathbf{x}_n}{1+\exp\{y_n \mathbf{w}'\mathbf{x}_n\}} = 0,
\end{aligned}$$

which does not have any closed form solution. However, because the problem is convex this can be solves globally using *gradient descent*. The only caveat is the case where data is perfectly linearly seperable. Then $Q(\mathbf{w}) \to 0$ as $\mathbf{w} \to \infty$ and the problem has no solution. Instead one can solve the L2-norm regularized problem (just like in linear regression) on the form

$$\hat{\mathbf{w}}_{MLE,L2} = \arg\min_{\mathbf{w}\in\mathbb{R}^{d+1}} Q(\mathbf{w}) + \lambda ||\mathbf{w}||_2^2,\ \lambda > 0,$$

which is a stricly convex problem.

4.1.2 *Multiple classification*

Defining the probability distribution: The only difference is how the probability distribution is defined. The MLE machinery + gradient descent always stays the same. For each category

$$\mathcal{Y} \in \{\mathcal{C}_1, \ldots, \mathcal{C}_m\},\ 2 \leq m$$

we define

$$\mathbb{P}\left(Y = y \mid X = \mathbf{x}; \mathbf{w}_1, \ldots, \mathbf{w}_m\right) \equiv \frac{\exp\{\delta_k(\mathbf{x})\}}{\sum_{j=1}^{m} \exp\{\delta_j(\mathbf{x})\}} = \frac{\exp\{\mathbf{w}_k'\mathbf{x}\}}{\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}\}}.$$

Log likelihood: This yields the nogative log likelihood function

$$\begin{aligned}
-\ln L(\mathbf{w}_1, \ldots, \mathbf{w}_m) &= -\ln \prod_{n=1}^{N} \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}_1, \ldots, \mathbf{w}_m\right) \\
&= -\sum_{n=1}^{N} \ln \mathbb{P}\left(Y = y_n \mid X = \mathbf{x}_n; \mathbf{w}_1, \ldots, \mathbf{w}_m\right) \\
&= -\sum_{n=1}^{N} \ln \frac{\exp\{\mathbf{w}_k'\mathbf{x}_n\}}{\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}_n\}}.
\end{aligned}$$

Identification: Since adding any vector $\mathbf{a} \in \mathbb{R}^{d+1}$ to $\mathbf{w}_k$, $i = 1, \ldots, m$ does not change the (negative log) likelihood:

$$\frac{\exp\{(\mathbf{w}_k + \mathbf{a})'\mathbf{x}_n\}}{\sum_{j=1}^{n} \exp\{(\mathbf{w}_j + \mathbf{a})'\mathbf{x}_n\}} = \frac{\exp\{\mathbf{a}\}\exp\{\mathbf{w}_k'\mathbf{x}_n\}}{\exp\{\mathbf{a}\}\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}_n\}} = \frac{\exp\{\mathbf{w}_k'\mathbf{x}_n\}}{\sum_{j=1}^{n} \exp\{\mathbf{w}_j'\mathbf{x}_n\}},$$

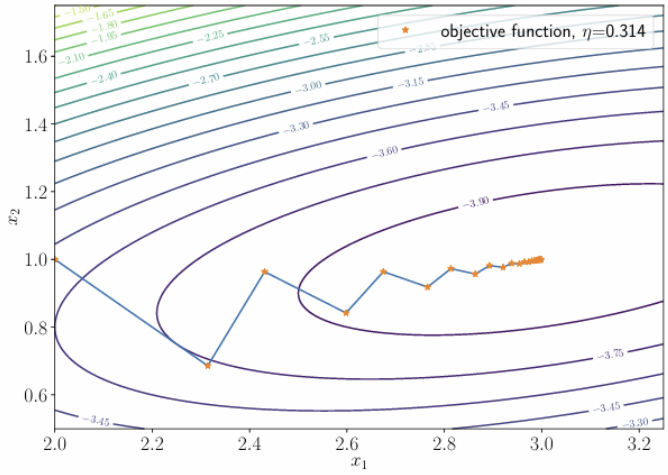
then we either need to add a regularization term or normalise one of the class-weights. For the latter solution a common choice is $\mathbf{w}_m = \mathbf{0} \in \mathbb{R}^{d+1}$. The remaining model-weights $\hat{\mathbf{w}}_1, \ldots, \hat{\mathbf{w}}_{m-1}$ are solved for using gradient descent.

4.1.3 *Optimization: Gradient descent methods*

Gradient descent: In short gradient descent iteratively updates the weights using a first order taylor expansion around the current point $\mathbf{w}^{(t)}$, where (t) indicates the iteration number. The goal is to minimize $Q(\mathbf{w})$.

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t)} \\ &= \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} Q|_{\mathbf{w}^{(t)}},\end{aligned}$$

with learning rate $\eta > 0$ determining how large steps are in proportion to the gradient.



Often we add a *momentum term* (in an attempt to improve performance) which intuitively works like a heavy ball. When a heavy ball has gained momentum from accelerating down a steep hill it will roll over local minimums and further down towards the global minimum (hopefully). Formally we have

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t-1)} \\ &= \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} Q|_{\mathbf{w}^{(t)}} + \mu \left(\mathbf{w}^{(t)} - \mathbf{w}^{(t-1)} \right),\end{aligned}$$

with momentum parameter $\mu > 0$.

Stochastic gradient descent: Esentially the same algorithm but where the gradi-

ent is calculated using one $(\mathbf{x}_n, y_n) \in S$, where S is the training data.

Minibatch gradient descent: The same but where the gradient is calculated on a random subset $S' \subset S$.

5 Random Forest

5.1 Binary decision trees (definitions)

CART trees are introduced and used in MLA and are popular binary classification / regression trees. Each *inner node* in a CART tree evaluates one dimension $d \in \{1, \ldots, D\}$ of the dataset using a threshold value θ and splits the data into

$$L_{d,\theta} = \{(\mathbf{x}, y) \in S \mid x_d < \theta\} \text{ and } R_{d,\theta} = \{(\mathbf{x}, y) \in S \mid x_d \geq \theta\},$$

where x_d is element d of $\mathbf{x}$. The subset $L_{d,\theta}$ is then sent to the left daughter node and $R_{d,\theta}$ to the right.

The splits are determined by maximising the information gain or equivalently minimising the impurity measure Q at the dataset at that note S. Formally we would like to choose the dimension d as well as the threshold value θ solving

$$\begin{aligned} &\arg \max_{d,\theta} Q(S) - \frac{|L_{d,\theta}|}{|S|} Q(L_{d,\theta}) - \frac{|R_{d,\theta}|}{|S|} Q(R_{d,\theta}) \\ = &\arg \min_{d,\theta} G_{d,\theta}(S) = |L_{d,\theta}| Q(L_{d,\theta}) + |R_{d,\theta}| Q(R_{d,\theta}). \end{aligned}$$

This is however impractical (and maybe even impossible) to solve on a computer and we therefore use some heuristic.

Leaf nodes are indexed by $\tau = 1, \ldots, |T|$ and they define *regions*. If $\mathbf{x}_i$ reaches leaf node τ we have $\mathbf{x} \in \mathcal{R}_\tau$.

5.1.1 Regression trees

Trees can not only classify observations $\mathbf{x}_i$ into categories but also estimate its value (if the label is continuous)

$$\hat{y}_i = T(\mathbf{x}_i) = \sum_{\tau=1}^{|T|} c_\tau \mathbb{I}(\mathbf{x}_i \in \mathcal{R}_\tau)$$

With squared loss $(\hat{y} - y)^2$ the constants minimising empirical risk are the respective empirical means of each leafnode τ:

$$c_\tau = \frac{1}{N_\tau} \sum_{\mathbf{x}_n \in \mathcal{R}_\tau} y_n,$$

with $(\mathbf{x}_n, y) \in S$ and $N_\tau = |\{(\mathbf{x}_n, y) \in S \mid \mathbf{x}_n \in \mathcal{R}_\tau\}|$.

5.1.2 Classification trees

For K classes, let $p_{\tau k}$ be the fraction of training observations in leaf τ with label k. The tree predicts the most frequent class in the leaf,

$$T(\mathbf{x}) = \underset{k \in \{1,\ldots,K\}}{\arg\max}\, p_{\tau k}, \qquad \mathbf{x} \in \mathcal{R}_\tau.$$

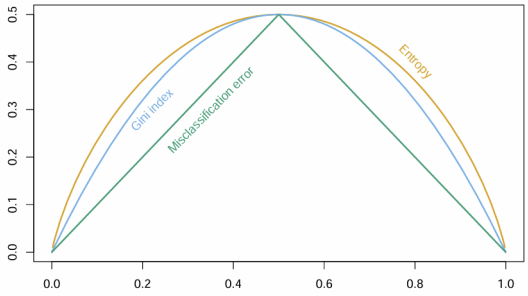
The fractions $p_{\tau k}$ also give estimated class probabilities. Common impurity measures are

$$Q_{\text{error}} = 1 - \max_k p_{\tau k},$$

$$Q_{\text{entropy}} = -\sum_{k=1}^{K} p_{\tau k} \log p_{\tau k},$$

$$Q_{\text{Gini}} = \sum_{k=1}^{K} p_{\tau k}(1 - p_{\tau k}),$$

where $0 \log 0 = 0$. All three are zero when the node is pure.



5.1.3 *Growing and pruning*

For each dimension d, it is enough to test thresholds halfway between consecutive distinct values of x_d. We split recursively until the node is pure, too small or has no valid split. For regression, Q is the mean squared deviation from the node mean.

For a single CART tree, we can prune by choosing a subtree T that minimises

$$C_\alpha(T) = \sum_{\tau=1}^{|T|} N_\tau Q_\tau(T) + \alpha|T|, \qquad \alpha \geq 0,$$

where $Q_\tau(T)$ is the impurity in leaf τ. Larger α penalises more leaves. We choose α using held-out data.

5.2 Ensembles and bagging

Single trees have high variance, so small changes in the training data can give very different trees. Bagging trains trees on separate bootstrap samples and averages their predictions (or votes).

If B regression trees have mean-zero errors ϵ_b with variance v and pairwise covariance c, then

$$\mathbb{E}\left[\left(\frac{1}{B}\sum_{b=1}^{B}\epsilon_b\right)^2\right] = \frac{v}{B} + \frac{B-1}{B}c.$$

Thus averaging helps more when the trees make less correlated errors. For binary classification, if errors are independent and each tree errs with probability at most $p_{\max} < 1/2$, Hoeffding bounds the majority vote error by

$\exp[-2B(1/2 - p_{\max})^2]$.

5.3 Growing and evaluating a random forest

For each $b = 1, \ldots, B$ we draw a bootstrap sample S_b by sampling N observations with replacement from S. We grow an unpruned tree T_b on S_b. At each node we draw a fresh subset of m distinct dimensions and find the best split among these. This makes the trees less correlated. With $m = D$ we recover bagging of trees.

For regression we average the predictions. For classification we take the most votes, or average the leaf probabilities and take the most probable class. Formally, these three rules are

$$
\begin{aligned}
f_{\mathrm{RF}}^{\mathrm{reg}}(\mathbf{x}) &= \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}), \\
f_{\mathrm{RF}}^{\mathrm{vote}}(\mathbf{x}) &= \arg\max_{k} \sum_{b=1}^{B} \mathbb{I}\big(T_b(\mathbf{x}) = k\big), \\
f_{\mathrm{RF}}^{\mathrm{prob}}(\mathbf{x}) &= \arg\max_{k} \frac{1}{B} \sum_{b=1}^{B} p_k^{(b)}(\mathbf{x}),
\end{aligned}
$$

where $p_k^{(b)}(\mathbf{x})$ is the fraction of class k in the leaf of T_b reached by $\mathbf{x}$.

Typical starting values are $m = \lfloor \sqrt{D} \rfloor$ for classification, $m = \max\{1, \lfloor D/3 \rfloor\}$ for regression and $B = 100$ trees.

5.4 Out-of-bag (OOB) error

An observation i is out of bag for tree b if its index was never drawn for S_b. This happens with probability

$$
\left(1 - \frac{1}{N}\right)^N \approx e^{-1} \approx 0.37.
$$

Let $\mathcal{B}_i$ be the set of trees for which i is out of bag. We predict $\mathbf{x}_i$ using the usual forest rule restricted to these trees. With $\mathcal{I} = \{i : \mathcal{B}_i \neq \varnothing\}$, the OOB estimate of

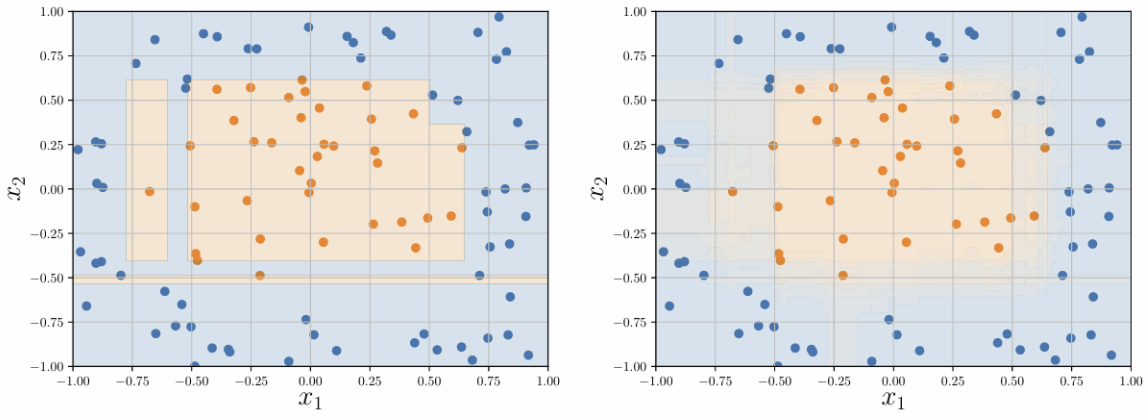
generalisation error is

$$\widehat{R}_{\mathrm{OOB}} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \ell\big(y_i, f_{\mathrm{RF}}(\{T_b : b \in \mathcal{B}_i\}, \mathbf{x}_i)\big).$$

We can choose m using OOB error and increase B until it stabilises.

5.5 Discussion: CART vs Random Forest

Single trees are easy to interpret but are unstable and have piecewise constant predictions. Random forests usually perform well on tabular data with little tuning, and the trees can be trained in parallel. The forest is harder to interpret and still uses axis-aligned splits. Class probabilities may be poorly calibrated.

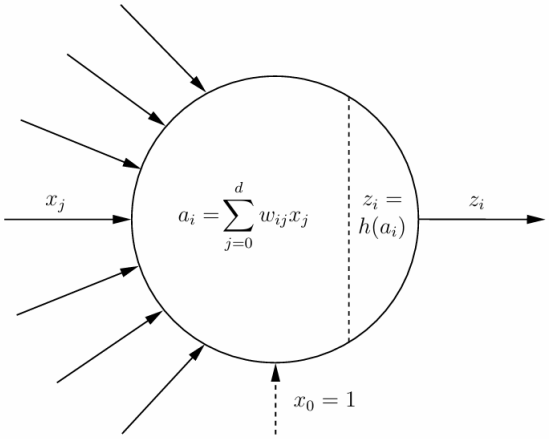


tree (left) vs. random forest (right)

6 Neural Networks

6.1 The Neuron

This chapter explains in detail the following picture which illustrates a single neuron with implicit bias. So if one understands the picture the chapter can be skipped.



6.1.1 *Pre-Activation*

The preactivation of neuron i is given by

$$\begin{aligned} a_i(\mathbf{x}) &= \sum_{j=1}^{d} w_{ij} x_j + w_{i0} \\ &= \mathbf{w}_i' \mathbf{x}_{+1}, \end{aligned}$$

where $\mathbf{x}_{+1} \in \mathbb{R}^{d+1}$ denotes the argmentet vector

$$\mathbf{x}_{+1} = \begin{bmatrix} 1 & x_1 & \cdots & x_d \end{bmatrix}'.$$

6.1.2 *Activation and output*

Applying a (non-linear) activation function $h_i : \mathbb{R} \to \mathbb{R}$ yields the output of neuron i

$$z_i(\mathbf{x}) = h_i(a_i(\mathbf{x})) = h_i \left(\sum_{j=1}^{d} w_{ij} x_j + w_{i0} \right).$$

That is each neuron z_i has its own weights (and bias) $\mathbf{w} \in \mathbb{R}^{d+1}$ and activation function h_i. The latter is usually the same for all neurons except the output-neurons (in the top layer).

6.1.3 *Activation functions*

Linear activation function: For the network structure to have any reason to exist the activation functions h_i must be non-linear. If not the neural network $f : \mathbb{R}^d \to \mathbb{R}^K$ will simply be a linear function of the inputs aswell.

To see this note that each neuron in the inputlayer would output the following functional form

$$\begin{aligned} h\left(\sum_{j=1}^{d} w_{ij}x_j + w_{i0}\right) &= \sum_{j=1}^{d} \underbrace{aw_{ij}}_{\tilde{w}_{ij}} x_j + \underbrace{aw_{i0} + b}_{\tilde{w}_{i0}} \\ &= \sum_{j=1}^{d} \tilde{w}_{ij}x_j + \tilde{w}_{i0}, \end{aligned}$$

and since each pre-activation is simply a affine linear function of the previous

affine linear functions all later neurons will output something of the form:

$$
\begin{aligned}
h\left(a\left(\begin{bmatrix} z_1 & z_2 & \cdots & z_K \end{bmatrix}\right)\right) &= h\left(\sum_{k=1}^{K} w_k z_k + w_0\right) \\
&= \sum_{k=1}^{K} \hat{w}_k z_k + \hat{w}_0 \\
&= \sum_{k=1}^{K} \hat{w}_k \left(\sum_{j=1}^{d} \tilde{w}_{kj} x_j + \tilde{w}_{k0}\right) + \hat{w}_0 \\
&= \sum_{k=1}^{K} \hat{w}_k \sum_{j=1}^{d} \tilde{w}_{kj} x_j + \underbrace{\sum_{k=1}^{K} \hat{w}_k \tilde{w}_{k0} + \hat{w}_0}_{\bar{w}_0} \\
&= \sum_{k=1}^{K} \sum_{j=1}^{d} \hat{w}_k \tilde{w}_{kj} x_j + \bar{w}_0 \\
&= \sum_{j=1}^{d} \sum_{k=1}^{K} \hat{w}_k \tilde{w}_{kj} x_j + \bar{w}_0 \\
&= \sum_{j=1}^{d} x_j \underbrace{\sum_{k=1}^{K} \left(\hat{w}_k \tilde{w}_{kj}\right)}_{\bar{w}_j} + \bar{w}_0 \\
&= \sum_{j=1}^{d} \bar{w}_j x_j + \bar{w}_0.
\end{aligned}
$$

This then also goes for all other layers recieving these outputs (by the exact same argument as above). So in summary: if h_i is linear for all i then there is no purpose to the neural network structure and the simple affine linear model of linear regression fits the model just as well.

Well known activation functions (greatest hits). Here i list some of the most common activation functions. Some are more common for outputlayers for example logistic activation and are rarely used in hidden layers.

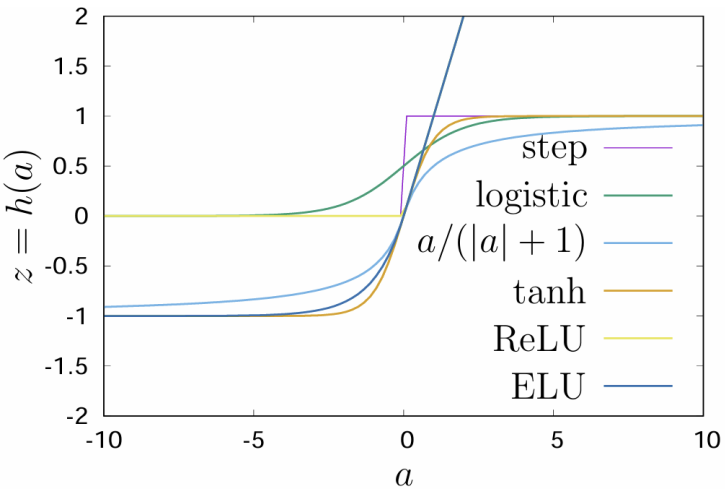
- Step / Threshold

  $$h(a) = \mathbb{I}(a > 0),$$

  with range $\mathcal{R} \in \{0, 1\}$

- Hyperbolic tangent
$$h(a) = tanh(a),$$
with range $\mathcal{R} \in (-1, 1)$ and with a visual form much like *alternative sigmoid.*
- Rectified linear unit (ReLU)
$$h(a) = \max(0, a),$$
with range $\mathcal{R}[0, \infty)$. Note that there the derivative of the loss function wrt. pre-activation weights is uninformative in the regrion $a < 0$. By that i mean $\partial h_i / \partial w_{ij} = 0$ if $a_i(\mathbf{x}, \mathbf{w}) < 0$ and so it does not help us adjust weights to decrease loss in these negative regions.
- Fermi / logistic
$$h(a) = \frac{1}{1 + \exp\{-a\}},$$
with range $\mathcal{R} \in (0, 1)$.
- Alternative sigmoid
$$h(a) = \frac{a}{1 + |a|},$$
with range $\mathcal{R}$
- Exponential LU (ELU, $\alpha > 0$)
$$h(a) = \begin{cases} a & a \geq 0 \\ \alpha(\exp\{a\} - 1) & a < 0 \end{cases}$$
with range $\mathcal{R} \in (-\alpha, \infty)$.

Here the functions are plottet where for ELU $\alpha = 1$ is chosen.

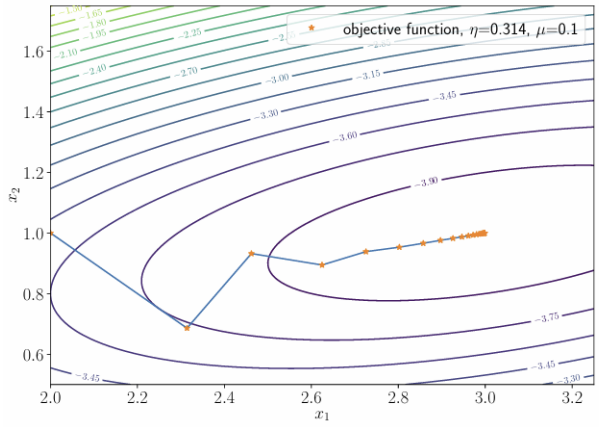


6.2 Gradient descent

To optimize a neural network with error function $E = \sum_{n=1}^{N} E_n$ (as with loglikelihood functions) we introduce gradient descent. The bare form of this algorithm is simply

$$\begin{aligned} \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t)} \\ &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}}, \end{aligned}$$

with learning rate $\eta > 0$ determining how large steps are in proportion to the gradient.



Often we add a *momentum term* (in an attempt to improve performance) which

intuitively works like a heavy ball. When a heavy ball has gained momentum from accelerating down a steep hill it will roll over local minimums and further down towards the global minimum (hopefully). Formally we have

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t-1)} \\ &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \left(\mathbf{w}^{(t)} - \mathbf{w}^{(t-1)} \right),\end{aligned}$$

with momentum parameter $\mu > 0$.

The question now really is how to calculate the elements of $\nabla E|_{\mathbf{w}^{(t)}}$ given by

$$\frac{\partial E}{\partial w_{ij}} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial w_{ij}},$$

for $(j, i) \in \mathcal{A}$. Recall that $\mathcal{A}$ contains all the edges connection the output of neuron j to the input of neuron i which is scaled by w_{ij}. This is answered in the next section.

6.3 Backpropagation

The goal is to calculate $\frac{\partial E_n}{\partial w_{ij}}$ for every $(j, i) \in \mathcal{A}$.[3] Applying the chain rule to every element of the error function $E = \sum_{n=1}^{N} E_n$ yields the following. Note that E_n is a function of z_i for all i in the network. and that any z_i is a function of z_j where $(j, i) \in \mathcal{A}$. Therefore the chain rule yields

$$\begin{aligned}\frac{\partial E_n}{\partial w_{ij}} &= \frac{\partial E_n}{\partial z_i} \frac{\partial z_i}{\partial a_i} \frac{\partial a_i}{\partial w_{ij}} \\ &= \frac{\partial E_n}{\partial z_i} h_i'(a_i) \left[\mathbf{z}_n\right]_j\end{aligned}$$

[3] Why not for $(j, i) \neq \mathcal{A}$? These weights simply do not exist. The model only has weights symbolizing the strength of the connect from neuron j to neuron i. There is no connection from i back to j so this w_{ji} does not exist.

where the last step uses that

$$z_i(\mathbf{z}) \equiv h_i(a_i(\mathbf{z})) \implies \frac{\partial z_i}{\partial a_i} = h_i'(a_i),$$

where $\mathbf{z} \in \mathbb{R}^M$ contains all the inputs from the previous layer to neuron i. For the last factor we used that

$$a_i(\mathbf{z}_n) = w_{i0} + \sum_{m=1}^{M} w_{im} \left[\mathbf{z}_n\right]_m \implies \frac{\partial a_i}{\partial w_{ij}} = \left[\mathbf{z}_n\right]_j.$$

Therefore the missing factor to be calculated is $\frac{\partial E_n}{\partial z_i}$ which we will now turn to.

6.3.1 The ouput layer

For the output layer $\frac{\partial E_n}{\partial z_i}$ can be directly calculated. To see this let the loss function be the sum of squared errors

$$\begin{aligned} E = \frac{1}{2}\sum_{n=1}^{N} ||f(\mathbf{x}_n;\mathbf{w}) - \mathbf{y}_n||^2 &= \frac{1}{2}\sum_{n=1}^{N}\left(\sqrt{\sum_{k=1}^{K}\left(\left[f(\mathbf{x}_n;\mathbf{w})\right]_k - \left[\mathbf{y}_n\right]_k\right)^2}\right)^2 \\ &= \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(\left[f(\mathbf{x}_n;\mathbf{w})\right]_k - \left[\mathbf{y}_n\right]_k\right)^2 \\ &= \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(\left[\mathbf{z}_n\right]_k - \left[\mathbf{y}_n\right]_k\right)^2 \end{aligned}$$

for $k \in V_{output}$. Therefore we have (for $i \in V_{output}$ and $(j,i) \in \mathcal{A}$)

$$\begin{aligned} \frac{\partial E_n}{\partial z_i} &= \frac{\partial}{\partial z_i}\frac{1}{2}\sum_{k=1}^{K}\left(\left[\mathbf{z}_n\right]_k - \left[\mathbf{y}_n\right]_k\right)^2 \\ &= \left[\mathbf{z}_n\right]_i - \left[\mathbf{y}_n\right]_i \end{aligned}$$

For $h_i(a_i) = a_i \implies h_i'(a_i) = 1$ where $i \in V_{output}$ we thus have

$$\begin{aligned}\frac{\partial E}{\partial w_{ij}} &= \sum_{n=1}^{N} \frac{\partial E_n}{\partial z_i} h_i'(a_i) z_j \\ &= \sum_{n=1}^{N} \frac{\partial E_n}{\partial z_i} \left[\mathbf{z}_n\right]_j \\ &= \sum_{n=1}^{N} \left(\left[\mathbf{z}_n\right]_i - \left[\mathbf{y}_n\right]_i\right) \left[\mathbf{z}_n\right]_j\end{aligned}$$

6.3.2 Hidden layers

For hidden layers we apply the apply the chainrule to obtain

$$\begin{aligned}\frac{\partial E}{\partial z_j} &= \sum_{k \text{ in } (j,k) \in \mathcal{A}} \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial z_j} \\ &= \sum_{k \text{ in } (j,k) \in \mathcal{A}} \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial a_k} \frac{\partial a_k}{\partial z_j} \\ &= \sum_{k \text{ in } (j,k) \in \mathcal{A}} \frac{\partial E}{\partial z_k} h_k'(a_k) w_{kj}.\end{aligned}$$

The partial derivatives of the layer just before V_{ouput} is then calculated by pluging in $\frac{\partial E}{\partial z_k}$, $k \in V_{output}$ and $(j,k) \in \mathcal{A}$. Then the third layers uses the second layers calculated $\frac{\partial E}{\partial z_j}$ and so on. This is sometimes referred to as the backwards pass.

7 Deep Learning

7.1 Convolution / cross-correlation

A convolution is a filtering transformation applied to image-like data. Image-like in the sence that datapoints close to each other in the dataset structure somehow relate to each other - like two pixels are closely related in a picture.

Formally let $C(i,j)$ be the output image from applying a convolution operation (or cross correlation operation) to the pixel in image I at location (i,j).

Additionally C takes a kernel matrix K as input.

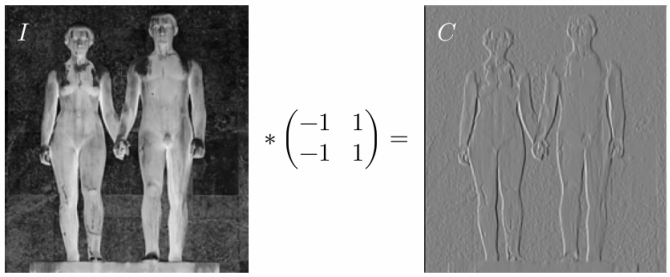
Cross correlation: It is intuitive and defined as

$$C_{corr}(i,j) = \sum_{m \in \mathcal{M}} \sum_{n \in \mathcal{N}} K(m,n) \cdot I(i+m, j+n),$$

where the kernel for example could be the vertical edge *filter kernel*:

$$K = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix},$$

and thus $\mathcal{M} = \{0,1\}$ and $\mathcal{N} = \{0,1\}$. So basically for each pixel we overlay the filter kernel K, weigh the surronding pixels accordingly and sum. An example is given here:



$$* \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix} =$$

Convolution: Essentially the same method but where we first flip K above 180-degrees, shiftet so the bottom right is at the pixel (i,j) and then apply cross correlation. The formula is

$$C_{conv}(i,j) = (I * K)(i,j) = \sum_{m \in \mathcal{M}} \sum_{n \in \mathcal{N}} K(m,n) \cdot I(i-m, j-n).$$

Note the shift dependents on $\mathcal{M}$ and $\mathcal{N}$. For example there is no shift for symmetrical convolutions

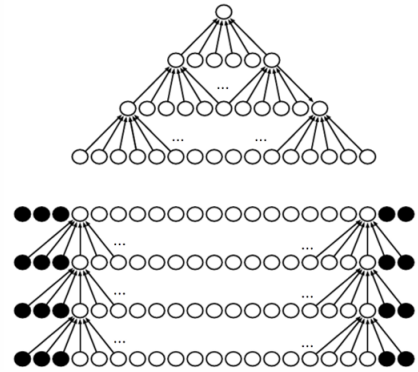
$$\mathcal{M} = \mathcal{N} = \{-k, -(k-1), \ldots, 0, \ldots, (k-1), k\}.$$

(Remember this is all in comparison with cross-correlation).

7.1.1 *Padding*

Convolution layers or cross correlation layers reduce the number of pixels. For example a 3 by 3 cross correlation will remove 2 pixels in each dimension thus reducing an image from (M, N) to $(M-2, N-2)$ to avoid this one can add *padding* around the input image I so that it becomes $(M+2, N+2)$.

The padding values are usually just zeros (zero padding) since the specific values rarely matters in practice. But it could also be some notion of mean values.



Goodfellow, Bengio, Courville. *Deep Learning*, 2016

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

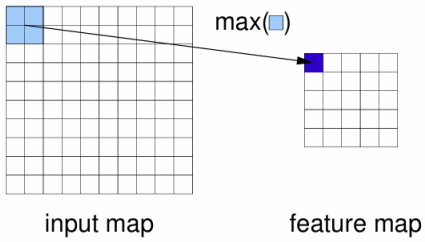
Bishop, Bishop. *Deep Learning*, 2024

7.1.2 *Equivariance*

If the input (for example a traffic sign in a picture) is translated / moved then the output (i.e. the feature space of the neurons in a particular layer) will translate in the same way. This is why convolutional neural networks are particularly relevant for image recognition tasks since it usually does not matter where in the frame a person or item is located.

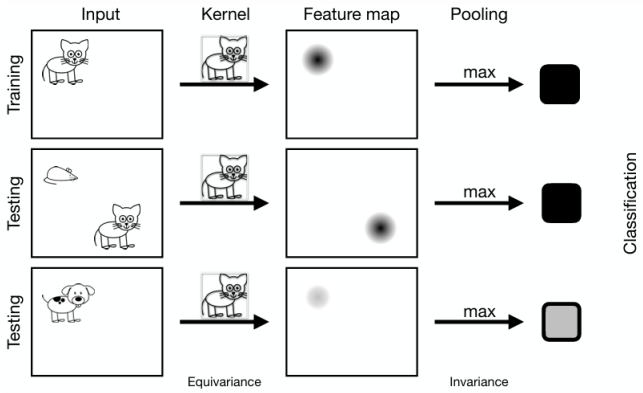
7.2 Pooling layer

Pooling layers reduce dimensionality by for example looping over all non-intersecting 2×2 grids on the feature image with a *stride* of 2. Then applying the $\max\{\cdot\}$ operator over those four values as the output. Thus in this example a 4 by 4 feature map is reduces to 2 by 2.



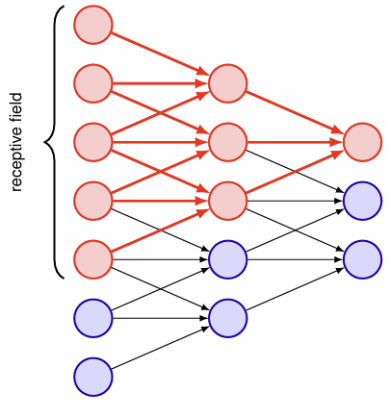
7.2.1 *Invariance*

Invariance refers to operations that does not change its output when the input is translated / shiftet. Pooling layers can help with invariance which can be understood through an example. Let the kernel detect cat-shapes. Then translating the cat 5 pixels to the right translates the feature map equivaraintly 5 pixels to the right (remember that the feature space is the output of the cross-correlation using the cat-kernel). Applying a max pooling layer will still just indicate `True` there is a cat. Thus the algorithm is *invariant* to translations of the cat.



7.2.2 *Receptive field*

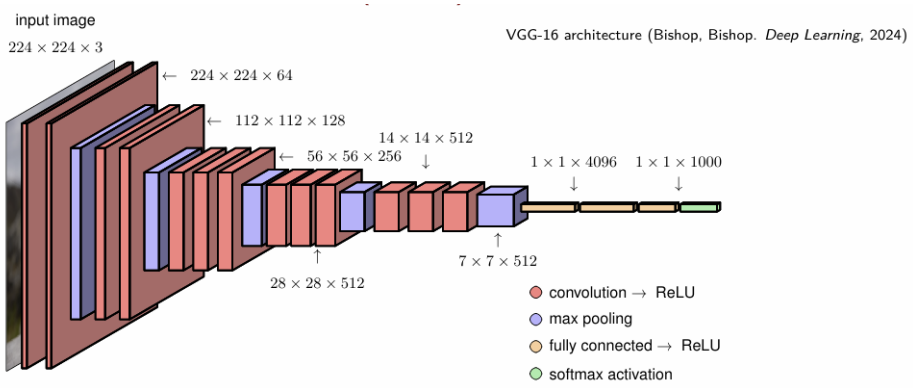
Nerons in deeper layers have a greater *effective receptive* field than its kernel would suggest. This is simply because the feature space the neuron percieves comes from a poolin layer which have condensed larger field.



Bishop, Bishop. *Deep Learning*, 2024

7.3 Example: VGG-16 architecture

As a quick recap I will explain the general details of the VGG-16 architecture depicted here.



The input layer is one RGB image - so there is 3 input channels corresponding to red, green and blue respectively. Each input channel is of size 244×244. The convolutional layer outputs 64 feature maps i.e. there are 64 kernels and biases needing to be trained in this single layer. Between each layer the non-linear ReLU activation function is applied. Each 224×224 feature-map is then reduced by max pooling to 122×122 and convolutional layers now take the 64 features and double them and creating 128 feature maps (thus and additional 128 kernels and biases needs training for each layer).

In the end, we obtain 512 feature maps, each of size 7×7. These are flattened

into a single vector containing $7 \cdot 7 \cdot 512 = 25,088$ features, which serves as input to a fully connected layer with 4096 neurons. This is followed by another fully connected layer with 4096 neurons and finally an output layer with 1,000 neurons, corresponding to the 1000 possible classification categories. A softmax activation is applied to the final output to obtain a probability distribution over these categories.

7.4 Tricks-of-the-Trade

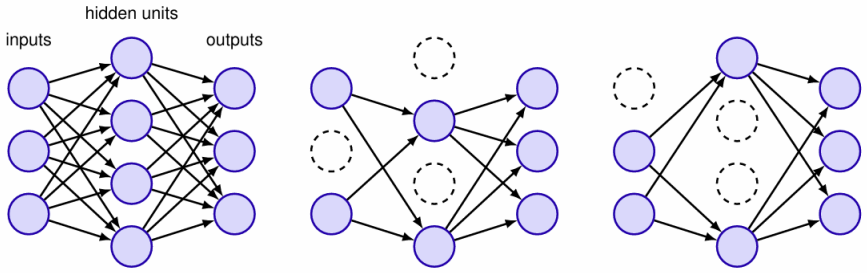
7.4.1 Ensemble Methods

Ensemble methods combat overfitting through the *cancellation of errors effect* and has proven to be very succesfull in the CNN setting. CNN ensembles can be created over the same datasets because of the importance of random initialization on the local minimum obtained.

Reminder: cancellation of errors effect: When individual ensemble members perform better than random guessing and make independent errors, their errors average out and the combined model outperforms the individual predictors.

7.4.2 Dropout

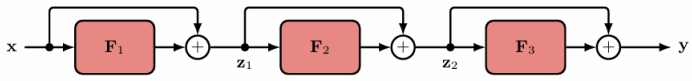
A heuristic method for avoiding overfitting is *dropout*. It is an ensemble method where each member of the ensemble is a subnetwork where randomly a binary mask **R** has been applied to remove neurons. Then training commences on all the subnetworks individually to create the ensemble. The two right models in the illustration would be two members of the whole ensemble.



Bishop, Bishop. *Deep Learning*, 2024

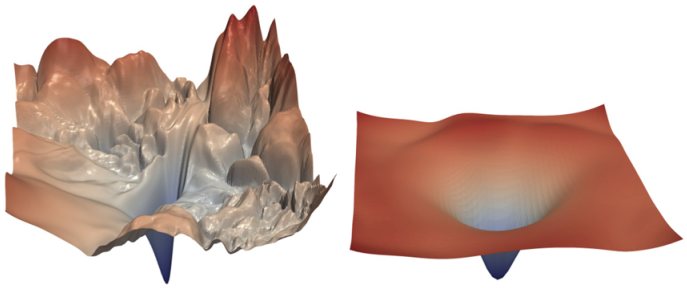
7.4.3 *Residual Leraning / Shorcuts*

Yet another method to combat local minima is shortcuts. Where in each layer the input data directly passes trough as illustrated below.



Bishop, Bishop. *Deep Learning*, 2024

This can drastically smooth the error surface. This example visualizes the error surface (loss vs. parameters) for a network with 56 layers without (a) and with (b) shortcuts.



(a) (b)

Li, Xu, Taylor, Studer, Goldstein. Visualizing the Loss Landscape of Neural Nets, *NeurIPS*, 2018

7.4.4 *Augmentation og Invariance*

Data augmentation refers to applying transformations to the input data that do not change their labels. Let $T = \{T1, \ldots, \}$ be a set of transformations. During training, for each

element added to a mini-batch it is decided if and which transformations are applied to it. In this way, we learn invariance with respect to the transformations in T. If augmentation is always applied, the network hardly see exactly the same input twice. Note that the "new" training examples are not i.i.d. samples!

Examples of transformations for images could be cropping, rotating, flipping, blurring, adding noise, ...

7.4.5 Batch and Layer Normalization

Batch normalization and layer normalization normalize intermediate neuron activations across different dimensions. This can stabilize gradient propagation, improve the conditioning of the optimization problem, and allow gradient descent to converge more reliably and often faster.

- Batch normalization normalizes *each* neurons activation over *multiple* observations (i.e. the batch).
- Layer normalization normalizes neuron activation *layer* for *each* observation.

III | Unsupervised Learning

8 Principal Component Analysis

8.1 Definitions

8.1.1 Dimensions

- D is the dimensionality of the original dataset X (sampled from $\mathcal{X}$). As an example consider the face database where each picture is 92×122 pixels. Then the dimension D of the input space is $D = 10304$.
- $d < D$ is the dimensionality of the outputspace of PCA i.e. the dimensionality we have reduced the dataset to using PCA.
- N is the number of observations in X
- X can then be defined by stacking all N observations $\mathbf{x}_i' \in \mathbb{R}^D$ as the rows to obtain $X \in \mathbb{R}^{N \times D}$. This is very much like the design matrix in linear regression).
- $U_d \in \mathbb{R}^{D \times d}$ takes in d dimensional enocded data and outputs D dimensional reconstructed data (where we add $\mathbf{b} \in \mathbb{R}^D$ for the final output $f_{dec}(\mathbf{z}_i) = \mathbf{b} + U_d \mathbf{z}_i$).

8.2 The objective of PCA

The goal of PCA is to take a dataset X sampled from$\mathcal{X}$ with datapoints $\mathbf{x}_i \in \mathbb{R}^D$, $i = 1, \ldots,$ and linearly map them into $\mathbf{z}_i \in \mathbb{R}^d$ where $d < D$ while minimizing the reconstruction error

$$J = \frac{1}{N} \sum_{n=1}^{N} ||\mathbf{x}_n - f_{dec}(\mathbf{z}_n)||_2^2,$$

where

$$f_{dec}(\mathbf{z}_n) = \mathbf{b} + U_m \mathbf{z}_n,$$

where $\mathbf{z}_i \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^D$ and $U_d \in \mathbb{R}^{D \times d}$ is the *decoding* function.

In other words we are looking for the *encoding* $f_{enc} : \mathbb{R}^D \to \mathbb{R}^d, d < D$

$$\min_{\mathbf{b}, U_d \{\mathbf{z}_1, \ldots, \mathbf{z}_N\}} \frac{1}{N} \sum_{n=1}^{N} ||\mathbf{x}_n - f_{dec}(\mathbf{z}_n)||_2^2$$

8.3 Optimal reconstruction

Let U (with the columns $\mathbf{u}_1, \ldots, \mathbf{u}_D \in \mathbb{R}^D$) be an orthonomal basis (spanning the the whole $\mathbb{R}^D$ with unit lenght basisvectors)

$$\mathbf{u}_k' \mathbf{u}_j = \begin{cases} 1 & k = j, \\ 0 & k \neq j. \end{cases}$$

Then every vector / observation $\mathbf{x}_i \in \mathbb{R}^D$ can be represented as a linear combination of the basisvectors in U

$$\mathbf{x}_i = \sum_{j=1}^{D} a_j \mathbf{u}_j. \tag{8.1}$$

We now want to determine a_j. Premultiply (8.1) by any of the new basisvectors $\mathbf{u}_k'$, $k \in \{1, \ldots, D\}$

$$\begin{aligned} \mathbf{u}_k' \mathbf{x}_i &= \mathbf{u}_k' \sum_{j=1}^{n} a_j \mathbf{u}_j = \sum_{j=1}^{n} \mathbf{u}_k' a_j \mathbf{u}_j = \sum_{j=1}^{n} a_j \mathbf{u}_k' \mathbf{u}_j \\ &= \underbrace{a_1 \mathbf{u}_k' \mathbf{u}_1 + \cdots +}_{=0} a_k \mathbf{u}_k' \mathbf{u}_k \underbrace{+ \cdots + a_\ell \mathbf{u}_k' \mathbf{u}_\ell}_{=0} \\ &= a_k, \end{aligned}$$

where we have used the pairwise orthogonality of the basis vectors and the unit length of any $\mathbf{u}_k$ such that $\mathbf{u}_k' \mathbf{u}_k = ||\mathbf{u}_k||_2^2 = 1^2$. Inserting $a_k = \mathbf{u}_k' \mathbf{x}_i$ for all $k \in \{1, \ldots, D\}$ into (8.1) yields

$$\mathbf{x}_i = \sum_{j=1}^{d} \left(\mathbf{u}_j' \mathbf{x}_i \right) \mathbf{u}_j.$$

Note that this only works because of the orthonormal restriction on the basis U.